

MoMo SMS Analytics

Database Design Document

Week 2 — Database Design and Implementation

Enterprise Fullstack Application

Team Members

Sano · Espoir · David · Gary · Merci

[Silicon Valley]

Submitted: May 18, 2026

Repository: github.com/SanoRod00/MOMO-sms-analytics

Table of Contents

1.	Executive Summary	3
2.	Entity Relationship Diagram	4
3.	Design Rationale and Justification	5
4.	Data Dictionary	6
5.	Sample Queries (CRUD Operations)	10
6.	Security and Integrity Rules	14
7.	AI Usage Acknowledgment	17
8.	Contributors	18

1. Executive Summary

This document presents the database design and implementation for the MoMo SMS Analytics system — an enterprise fullstack application that processes MTN Mobile Money SMS notifications, stores the transactions in a normalized relational database, and exposes analytical insights through a frontend dashboard.

The Week 2 scope covered three areas: designing an Entity Relationship Diagram from real MoMo SMS data, implementing the schema in MySQL with full referential integrity and performance optimization, and modeling the same data structures as JSON for future API responses.

Five normalized tables were designed: users, transaction_categories, transactions, transaction_participants, and system_logs. A junction table (transaction_participants) resolves the many-to-many relationship between transactions and users — a design choice that directly reflects the dual-party nature of mobile money transactions, where every transaction has at least a sender and a receiver.

The implementation in database_setup.sql creates all tables with proper data types, primary and foreign keys, indexes for performance, and meaningful column comments. Security and integrity rules layered on top (CHECK constraints, triggers, views) protect against invalid data and provide audit trails. The schema was validated through a complete suite of CRUD and analytical queries, the results of which are documented in this report.

2. Entity Relationship Diagram

The diagram below shows the five entities, their attributes, and the relationships connecting them. Primary keys are marked PK, foreign keys are marked FK, and crow's-foot notation indicates relationship cardinality. The junction table `transaction_participants` resolves the many-to-many relationship between transactions and users.

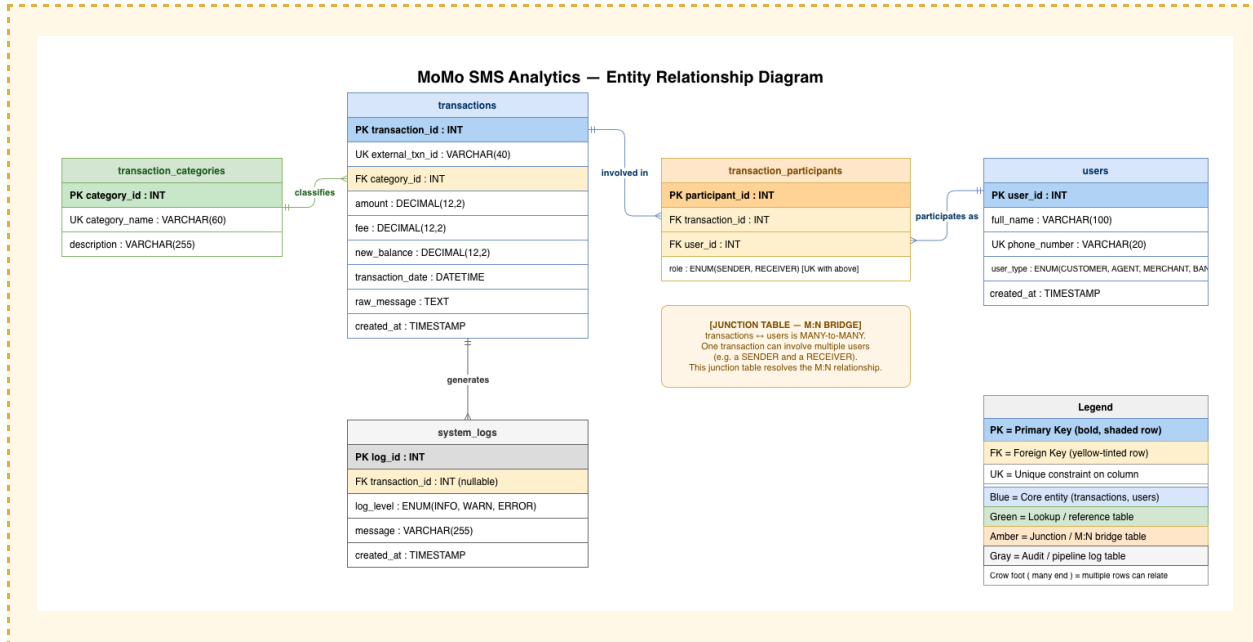


Figure 1: MoMo SMS Analytics — Entity Relationship Diagram

3. Design Rationale and Justification

This relational schema models the MoMo SMS data using five normalized tables to maintain data integrity, eliminate redundancy, and support efficient querying. The design separates concerns clearly: users describe people and entities, `transaction_categories` provide a controlled vocabulary, `transactions` store the core financial events, `transaction_participants` link users to transactions, and `system_logs` capture pipeline observability for processing audit.

The central design decision is modelling the relationship between transactions and users as many-to-many, resolved through the `transaction_participants` junction table. A single transaction always involves at least two users — a sender and a receiver — and one user participates in many transactions over time. Modelling this with two foreign keys (`sender_id`, `receiver_id`) directly on `transactions` would produce two separate one-to-many relationships, obscure the symmetry between participants, and complicate any future extension to multi-party transactions. The junction table makes the relationship explicit, enforces the role of each participant through an ENUM, and prevents duplicate participation through a UNIQUE constraint on (`transaction_id`, `user_id`, `role`).

Data types were chosen for accuracy and storage efficiency. DECIMAL(12,2) is used for all monetary fields because FLOAT introduces rounding errors that are unacceptable in financial data. DATETIME captures the precise transaction time read from the SMS body. ENUMs constrain `user_type`, `direction`, `role`, and `log_level` to a fixed vocabulary, preventing inconsistent free-text values. The `phone_number` column is intentionally nullable because real MoMo SMS messages frequently mask the counterparty's number (for example, *****013).

Referential integrity is enforced through FOREIGN KEY constraints, and indexes on foreign keys and on `transaction_date` support fast joins and time-range analytical queries. The `transaction_categories` lookup table prevents typos in category names, simplifies future analytics, and allows category labels to be updated across all transactions through a single row change. Together, these decisions yield a robust, query-efficient, and maintainable foundation that supports both the current ETL pipeline and future API or dashboard work.

4. Data Dictionary

This dictionary documents every table and column in the database. Each entry lists the column name, MySQL data type, constraints, and a description of its business meaning.

4.1 users

Represents every person or entity that participates in a transaction — the account holder, peer-to-peer counterparties (e.g. Jane Smith), merchants (e.g. DIRECT PAYMENT LTD), and pseudo-users for banks or the system itself.

Column	Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique identifier for each user
full_name	VARCHAR(100)	NOT NULL	Full name of the user as it appears in the SMS
phone_number	VARCHAR(20)	UNIQUE, NULL	Mobile number; nullable because real MoMo SMS frequently mask counterparty numbers
user_type	ENUM	NOT NULL	One of: customer, merchant, bank, system
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the row was inserted

4.2 transaction_categories

Lookup table of transaction types observed in the SMS data: Incoming Money, Payment to Code Holder, Transfer to Mobile, Bank Deposit, Airtime Purchase, Direct Payment, Withdrawal, Bundle Purchase, and so on.

Column	Type	Constraints	Description
category_id	INT	PK, AUTO_INCREMENT	Unique identifier for each category
category_name	VARCHAR(50)	UNIQUE, NOT NULL	Category name (e.g. Payment to Code Holder, Bank Deposit)
description	VARCHAR(255)	NULL	Human-readable description of what the category represents

4.3 transactions

The core entity: one row per mobile money transaction parsed from an SMS. Amount, fee, balance, and timestamp are extracted from the message body; the original raw_message is preserved for audit.

Column	Type	Constraints	Description
transaction_id	INT	PK, AUTO_INCREMENT	Unique identifier for each transaction

Column	Type	Constraints	Description
external_txn_id	VARCHAR(50)	UNIQUE, NULL	Txid or Financial Transaction Id extracted from the SMS body
category_id	INT	FK → transaction_categories	Category classifying the transaction
amount	DECIMAL(12,2)	NOT NULL, CHECK ≥ 0	Transaction amount in RWF
fee	DECIMAL(12,2)	DEFAULT 0, CHECK ≥ 0	Fee charged on the transaction
new_balance	DECIMAL(12,2)	NULL	Mobile money balance after the transaction
transaction_date	DATETIME	NOT NULL	Timestamp of the transaction parsed from the SMS body
direction	ENUM	NOT NULL	One of: credit, debit
raw_message	TEXT	NULL	Original SMS body preserved for audit and reprocessing
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the row was inserted

4.4 transaction_participants (junction table)

Resolves the many-to-many relationship between transactions and users. Each row records one user playing one role (sender or receiver) on one transaction. A transaction typically has exactly two participant rows.

Column	Type	Constraints	Description
participant_id	INT	PK, AUTO_INCREMENT	Unique identifier for each participant link
transaction_id	INT	FK → transactions	The transaction this participant belongs to
user_id	INT	FK → users	The user participating in the transaction
role	ENUM	NOT NULL	One of: sender, receiver
UNIQUE constraint	—	(transaction_id, user_id, role)	Prevents a user from being recorded twice in the same role on the same transaction

4.5 system_logs

Records observability events from the ETL pipeline and from triggers in the database itself. Used for debugging the import process and for auditing transaction inserts.

Column	Type	Constraints	Description
log_id	INT	PK, AUTO_INCREMENT	Unique identifier for each log entry
transaction_id	INT	FK → transactions, NULL	Linked transaction (nullable for pipeline-level logs)
log_level	ENUM	NOT NULL	One of: INFO, WARNING, ERROR

Column	Type	Constraints	Description
log_message	VARCHAR(500)	NOT NULL	Human-readable log message
processing_stage	VARCHAR(50)	NULL	Pipeline stage (parse, clean, categorize, load)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the log was written

5. Sample Queries (CRUD Operations)

The following queries demonstrate that the database supports all four CRUD operations correctly, plus a set of analytical queries that prove the design is fit for downstream insights. Each query was run against the seeded data from `database_setup.sql`; result screenshots follow each query.

5.1 Create (Insert)

5.1.1 Insert a new merchant user

Purpose: Add a new merchant to the users table.

```
INSERT INTO users (full_name, phone_number, user_type)
VALUES ('CITY MART RWANDA', '250788123456', 'merchant');
```

The screenshot shows the DB Fiddle interface with the following details:

- Database:** MySQL v8
- Schema SQL:**

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9 CREATE TABLE transaction_categories (
10  category_id INT AUTO_INCREMENT PRIMARY KEY,
11  category_name VARCHAR(60) NOT NULL UNIQUE,
12  description VARCHAR(255)
13 );
```
- Query SQL:**

```
1 -- 01: INSERT new merchant user
2 INSERT INTO users (full_name, phone_number, user_type)
3 VALUES ('Rwanda Coffee House', NULL, 'MERCHANT');
4
5 SELECT * FROM users;
```
- Results:**
 - Query #1: Execution time: 12.83ms. There are no results to be displayed.
 - Query #2: Execution time: 0.31ms. Results table:

user_id	full_name	phone_number	user_type	created_at
1	Samuel Carter	+250788111001	CUSTOMER	2026-05-20 16:03:20
2	Jane Smith	+250788222002	CUSTOMER	2026-05-20 16:03:20

Result: 1 row inserted; the new merchant is now available for future transactions.

5.1.2 Insert a new transaction

Purpose: Record a payment transaction referencing the existing 'Payment to Code Holder' category.

```
INSERT INTO transactions
  (external_txn_id, category_id, amount, fee, new_balance, transaction_date, direction,
  raw_message)
SELECT 'TX-DEMO-001', c.category_id, 2500.00, 0.00, 12500.00,
      '2024-05-15 14:30:00', 'debit', 'Test transaction for CRUD demo'
FROM transaction_categories c
WHERE c.category_name = 'Payment to Code Holder';
```

Prep for your next tech interview with Exponent →

Database: MySQL v8 Run Save Load Example Collaborate Sign in Have any feedback?

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

Carbon

What makes you click on a dev tool ad? Honest answers only. 3-min

Schema SQL

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER','AGENT','MERCHANT','BANK','SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9 CREATE TABLE transaction_categories (
10  category_id INT AUTO_INCREMENT PRIMARY KEY,
11  category_name VARCHAR(60) NOT NULL UNIQUE,
12  description VARCHAR(255)
13 );
```

Query SQL

```
1 -- 02: INSERT new transaction (Payment to Code Holder)
2 INSERT INTO transactions
3   (external_txn_id, category_id, amount, fee, new_balance,
4    transaction_date, raw_message)
5   VALUES (
6     'TX1006', 2, 7500.00, 75.00, NULL,
7     '2024-05-13 10:00:00',
8     'TxId: TX1006. Your payment of 7500 RWF to Rwanda Coffee House.'
9 );
10 SELECT * FROM transactions WHERE external_txn_id = 'TX1006';
```

Results

Query #1 Execution time: 0.75ms

There are no results to be displayed.

Query #2 Execution time: 0.28ms

transaction_id	external_txn_id	category_id	amount	fee	new_balance	transaction_date	raw_message	created_at
6	TX1006	2	7500.0	75.0	null	2024-05-13 10:00:00	Txid: TX1006. Your payment of 7500 RWF to Rwanda Coffee House.	2026-05-20 16:03:48

DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom. Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Result showing the new transaction inserted, including its generated transaction_id.

5.1.3 Link the transaction to its participants

Purpose: Create the two participant rows (sender and receiver) for the transaction inserted above.

```
INSERT INTO transaction_participants (transaction_id, user_id, role)
SELECT t.transaction_id, u.user_id, 'sender'
FROM transactions t, users u
WHERE t.external_txn_id = 'TX-DEMO-001' AND u.full_name = 'CITY MART RWANDA';
```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▶ Run Save Load Example Collaborate → Sign in Have any feedback? 🐦

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

design.dev
Design & Dev Tools
For Creators

Copy-paste ready CSS & JavaScript patterns with accessibility baked in.

Schema SQL

```

1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9 CREATE TABLE transaction_categories (
10  category_id INT AUTO_INCREMENT PRIMARY KEY,
11  category_name VARCHAR(60) NOT NULL UNIQUE,
12  description VARCHAR(255)
13 );

```

Text to DDL

Query SQL

```

1 -- 03: INSERT participants linking TX1006 to Samuel (SENDER) and Rwanda
  Coffee House (RECEIVER)
2 INSERT INTO transaction_participants (transaction_id, user_id, role)
3 VALUES
4   (6, 1, 'SENDER'),
5   (6, 6, 'RECEIVER');
6
7 SELECT tp.participant_id, tp.transaction_id, u.full_name, u.user_type,
8   tp.role
9 FROM transaction_participants tp
9 JOIN users u ON tp.user_id = u.user_id
10 WHERE tp.transaction_id = 6;

```

Results [Copy as Markdown](#)

Query #1 **Execution time: 0.77ms**

There are no results to be displayed.

Query #2 **Execution time: 0.38ms**

participant_id	transaction_id	full_name	user_type	role
11	6	Samuel Carter	CUSTOMER	SENDER
12	6	Rwanda Coffee House	MERCHANT	RECEIVER

DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom. [Terms of Use](#) • [Privacy](#) / [Cookie Policy](#) • Status200 Ltd © 2018

Rows successfully inserted into transaction_participants linking the transaction to its sender.

5.2 Read (Select)

5.2.1 All payments, most recent first

Purpose: List every transaction in the 'Payment to Code Holder' category, newest first.

```

SELECT t.transaction_id, t.amount, t.fee, t.transaction_date, c.category_name
FROM transactions t
JOIN transaction_categories c ON t.category_id = c.category_id
WHERE c.category_name = 'Payment to Code Holder'
ORDER BY t.transaction_date DESC
LIMIT 10;

```

Purpose: Find all transactions above 5,000 RWF.

Page 12

The screenshot shows the DB Fiddle interface with a MySQL database. The Schema SQL section contains the following DDL:

```

1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60) NOT NULL UNIQUE,
13   description VARCHAR(255)
14 );

```

The Query SQL section contains the following query:

```

1 -- 05: SELECT all transactions with amount > 5000 RWF
2 SELECT
3   t.external_txn_id,
4   tc.category_name,
5   t.amount,
6   t.transaction_date
7 FROM transactions AS t
8 JOIN transaction_categories AS tc ON t.category_id = tc.category_id
9 WHERE t.amount > 5000.00
10 ORDER BY t.amount DESC;

```

The Results section shows the output of the query:

external_txn_id	category_name	amount	transaction_date
TX1004	Bank Deposit	50000.0	2024-05-11 15:45:00
TX1001	Incoming Money	25000.0	2024-05-10 09:12:00
TX1003	Transfer to Mobile	10000.0	2024-05-11 08:05:00
TX1006	Payment to Code Holder	7500.0	2024-05-13 10:00:00

Below the screenshot, the text reads: *All high-value transactions, ordered by amount.*

5.2.3 Full transaction history for one user (joins through junction table)

Purpose: Show every transaction a specific user participated in, with their role. This query exercises the many-to-many design.

```

SELECT u.full_name, tp.role, t.transaction_id, c.category_name,
       t.amount, t.transaction_date
FROM users u
JOIN transaction_participants tp ON tp.user_id = u.user_id
JOIN transactions t              ON t.transaction_id = tp.transaction_id
JOIN transaction_categories c    ON c.category_id = t.category_id
WHERE u.full_name = 'Jane Smith'
ORDER BY t.transaction_date DESC;

```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▾ Run Save Load Example Collaborate → Sign in Have any feedback?

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

design.dev
Design & Dev Tools
For Creators

Copy-paste ready CSS & JavaScript patterns with accessibility baked in.

Schema SQL

```

1 CREATE TABLE users (
2   user_id      INT AUTO_INCREMENT PRIMARY KEY,
3   full_name    VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20)  UNIQUE,
5   user_type    ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9 CREATE TABLE transaction_categories (
10  category_id INT AUTO_INCREMENT PRIMARY KEY,
11  category_name VARCHAR(60) NOT NULL UNIQUE,
12  description  VARCHAR(255)
13 );

```

Text to DDL

Query SQL

```

1 -- 06: Samuel Carter's full transaction history (M:N join proof)
2 SELECT
3   t.external_txn_id,
4   tc.category_name,
5   tp.role AS user_role,
6   t.amount,
7   t.fee,
8   t.transaction_date
9 FROM transaction_participants AS tp
10 JOIN transactions AS t ON tp.transaction_id = t.transaction_id
11 JOIN transaction_categories AS tc ON t.category_id = tc.category_id
12 JOIN users AS u ON tp.user_id = u.user_id
13 WHERE u.full_name = 'Samuel Carter'
14 ORDER BY t.transaction_date;

```

Results

Query #1 Execution time: 10.68ms

external_txn_id	category_name	user_role	amount	fee	transaction_date
TX1001	Incoming Money	RECEIVER	25000.0	0.0	2024-05-10 09:12:00
TX1002	Payment to Code Holder	SENDER	4500.0	50.0	2024-05-10 12:30:00
TX1003	Transfer to Mobile	SENDER	10000.0	100.0	2024-05-11 08:05:00
TX1004	Bank Deposit	RECEIVER	50000.0	0.0	2024-05-11 15:45:00

Copy as Markdown

DB Fiddle - Crafted with by Status200 in the United Kingdom.

Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Jane Smith's full transaction history showing both sender and receiver roles — proves the M:N design works.

Result: Each row shows the user's role on a specific transaction, confirming that one user can play different roles across many transactions.

5.2.4 Count of transactions per category

Purpose: Aggregate the volume of transactions in each category.

```

SELECT c.category_name, COUNT(*) AS txn_count
FROM transactions t
JOIN transaction_categories c ON t.category_id = c.category_id
GROUP BY c.category_name
ORDER BY txn_count DESC;

```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▾ Run Save Load Example Collaborate Sign in Have any feedback?

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

Carbon

What makes you click on a dev tool ad? Honest answers only. 3-min

Schema SQL

```

1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60) NOT NULL UNIQUE,
13   description VARCHAR(255)
14 );

```

Text to DDL

Query SQL

```

1 -- 07: Count of transactions per category (GROUP BY)
2 SELECT
3   tc.category_name,
4   COUNT(t.transaction_id) AS transaction_count
5 FROM transaction_categories AS tc
6 LEFT JOIN transactions AS t ON tc.category_id = t.category_id
7 GROUP BY tc.category_id, tc.category_name
8 ORDER BY transaction_count DESC;

```

Results

Query #1 Execution time: 0.5ms

category_name	transaction_count
Payment to Code Holder	2
Airtime Purchase	1
Bank Deposit	1
Incoming Money	1

Copy as Markdown

DB Fiddle - Crafted with by Status200 in the United Kingdom. Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Transaction count per category in descending order — shows Payment and Transfer dominate the dataset.

5.3 Update

5.3.1 Reclassify a transaction

Purpose: Move a transaction from one category to another (used when categorization needs correction).

```

UPDATE transactions
SET category_id = (SELECT category_id FROM transaction_categories WHERE category_name =
'Airtime Purchase')
WHERE external_txn_id = 'TX-DEMO-001';

```

Prep for your next tech interview with Exponent →

Database: MySQL v8 Run Save Load Example Collaborate Sign in Have any feedback?

Fiddle Title
50 characters remaining.

Fiddle Description
300 characters remaining.

Private Fiddle **PRO**
This setting cannot be modified after saving the fiddle.

Upgrade to PRO
50% OFF for Early Adopters

Show Keyboard Shortcuts

Carbon 
What makes you click on a dev tool ad? Honest answers only. 3-min

Schema SQL

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER','AGENT','MERCHANT','BANK','SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60) NOT NULL UNIQUE,
13   description VARCHAR(255)
14 );
```

Text to DDL

Query SQL

```
1 -- 08: UPDATE TX1006 category from Payment to Code Holder -> Airtime
2   Purchase
3 UPDATE transactions
4 SET category_id = (
5   SELECT category_id FROM transaction_categories
6   WHERE category_name = 'Airtime Purchase'
7 )
8 WHERE external_txn_id = 'TX1006';
9
10 -- Verify the change
11 SELECT t.external_txn_id, tc.category_name, t.amount
12 FROM transactions AS t
13 JOIN transaction_categories AS tc ON t.category_id = tc.category_id
14 WHERE t.external_txn_id = 'TX1006';
```

Results

Query #1 Execution time: 0.82ms
There are no results to be displayed.

Query #2 Execution time: 0.22ms

external_txn_id	category_name	amount
TX1006	Airtime Purchase	7500.0

Copy as Markdown

DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom. Terms of Use • Privacy • Cookie Policy • Status200 Ltd © 2018

Result showing 1 row affected; transaction is now categorized as Airtime Purchase.

5.3.2 Correct a masked phone number

Purpose: Replace a previously-masked phone number with the full number once known.

```
UPDATE users
SET phone_number = '250788111222'
WHERE full_name = 'Jane Smith' AND phone_number IS NULL;
```

Prep for your next tech interview with Exponent →

Database: MySQL v8 Run Save Load Example Collaborate Sign in Have any feedback?

Fiddle Title
50 characters remaining.

Fiddle Description
300 characters remaining.

Private Fiddle **PRO**
This setting cannot be modified after saving the fiddle.

Upgrade to PRO
50% OFF for Early Adopters

Show Keyboard Shortcuts

Carbon 
What makes you click on a dev tool ad? Honest answers only. 3-min

Schema SQL

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER','AGENT','MERCHANT','BANK','SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60) NOT NULL UNIQUE,
13   description VARCHAR(255)
14 );
```

Text to DDL

Query SQL

```
1 -- 09: UPDATE MTN Agent phone number (fix masked/malformed value)
2 UPDATE users
3 SET phone_number = '+250788333099'
4 WHERE full_name = 'MTN Agent 54321';
5
6 -- Verify the change
7 SELECT user_id, full_name, phone_number, user_type
8 FROM users
9 WHERE full_name = 'MTN Agent 54321';
```

Results

Query #1 Execution time: 0.69ms
There are no results to be displayed.

Query #2 Execution time: 0.23ms

user_id	full_name	phone_number	user_type
5	MTN Agent 54321	+250788333099	AGENT

Copy as Markdown

DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom. Terms of Use • Privacy • Cookie Policy • Status200 Ltd © 2018

User record updated with the full phone number.

5.4 Delete

5.4.1 Delete a test transaction

Purpose: Remove the demo transaction inserted above (cleanup).

```
-- Before
SELECT * FROM transactions WHERE external_txn_id = 'TX-DEMO-001';

-- Delete
DELETE FROM transactions WHERE external_txn_id = 'TX-DEMO-001';

-- After
SELECT * FROM transactions WHERE external_txn_id = 'TX-DEMO-001';
```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▶ Run Save Load Example Collaborate → Sign in Have any feedback?

Fiddle Title
50 characters remaining.

Fiddle Description
300 characters remaining.

Private Fiddle **PRO**
This setting cannot be modified after saving the fiddle.

Upgrade to PRO
50% OFF for Early Adopters

Show Keyboard Shortcuts

Carbon

What makes you click on a dev tool ad? Honest answers only. 3-min

Schema SQL

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER','AGENT','MERCHANT','BANK','SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9 CREATE TABLE transaction_categories (
10  category_id INT AUTO_INCREMENT PRIMARY KEY,
11  category_name VARCHAR(60) NOT NULL UNIQUE,
12  description VARCHAR(255)
13 );
```

Text to DDL

Query SQL

```
1 -- 10: SELECT before DELETE - confirm TX1006 exists
2 SELECT t.transaction_id, t.external_txn_id, t.amount, t.transaction_date
3 FROM transactions AS t
4 WHERE t.external_txn_id = 'TX1006';
```

Results

Query #1 Execution time: 0.34ms

transaction_id	external_txn_id	amount	transaction_date
6	TX1006	7500.0	2024-05-13 10:00:00

Copy as Markdown

DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom. Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Before/after SELECT pair showing the row existed and is now gone.

5.4.2 Attempt to delete a referenced user

Purpose: Try to delete a user who still has participant rows — demonstrates how the FK constraint protects referential integrity.

```
DELETE FROM users WHERE full_name = 'Jane Smith';
```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▾ Run Save Load Example Collaborate Sign in Have any feedback? 

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

 Carbon

What makes you click on a dev tool ad? Honest answers only. 3-min

Schema SQL

```

1 CREATE TABLE users (
2   user_id      INT AUTO_INCREMENT PRIMARY KEY,
3   full_name    VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20)  UNIQUE,
5   user_type    ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11  category_id  INT AUTO_INCREMENT PRIMARY KEY,
12  category_name VARCHAR(60)  NOT NULL UNIQUE,
13  description  VARCHAR(255)
14 );

```

Text to DDL

Query SQL

```

1 -- 11: DELETE TX1006 (CASCADE removes its participant rows
  automatically)
2 DELETE FROM transactions WHERE external_txn_id = 'TX1006';
3
4 -- Verify transaction is gone (expect 0 rows)
5 SELECT transaction_id, external_txn_id FROM transactions WHERE
  external_txn_id = 'TX1006';

```

Results

Query #1 **Execution time: 0.93ms**

There are no results to be displayed.

Query #2 **Execution time: 0.19ms**

transaction_id	external_txn_id
There are no results to be displayed.	

Copy as Markdown

DB Fiddle - Crafted with  by Status200 in the United Kingdom. Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

MySQL error: cannot delete parent row because of foreign key constraint — confirms referential integrity is enforced.

5.5 Analytical Queries

5.5.1 Total amount transacted per category

Purpose: Sum the value of money flowing through each category.

```

SELECT c.category_name,
       SUM(t.amount) AS total_amount,
       COUNT(*)      AS txn_count,
       AVG(t.amount) AS avg_amount
FROM transactions t
JOIN transaction_categories c ON t.category_id = c.category_id
GROUP BY c.category_name
ORDER BY total_amount DESC;

```

The screenshot shows the DB Fiddle web application interface. At the top, there's a green banner with the text "Prep for your next tech interview with Exponent →". Below this is a blue header bar with navigation links: "Database: MySQL v8", "Run", "Save", "Load Example", "Collaborate", "Sign in", and "Have any feedback?".

The main interface is divided into three columns:

- Left Column:** Contains a "Fiddle Title" input field (50 characters remaining), a "Fiddle Description" input field (300 characters remaining), a "Private Fiddle" toggle switch (set to "PRO"), and an "Upgrade to PRO" button with a "50% OFF for Early Adopters" offer. At the bottom, there's a "Show Keyboard Shortcuts" link and a "design.dev" logo.
- Schema SQL:** Contains the following SQL code:


```
1 CREATE TABLE users (
2   user_id      INT AUTO_INCREMENT PRIMARY KEY,
3   full_name    VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20)  UNIQUE,
5   user_type    ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id  INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60)  NOT NULL UNIQUE,
13   description  VARCHAR(255)
14 );
```
- Query SQL:** Contains the following SQL code:


```
1 -- 12: Verify ON DELETE CASCADE -- no orphan participant rows after
2 TX1006 deletion
3 SELECT tp.participant_id, tp.transaction_id, tp.role
4 FROM transaction_participants AS tp
5 WHERE tp.transaction_id NOT IN (SELECT transaction_id FROM
6 transactions);
7 -- Expected: 0 rows (cascade worked correctly)
```

Below the SQL editor, there's a "Results" section. It shows "Query #1" with an "Execution time: 10.54ms". A table with the following structure is displayed:

participant_id	transaction_id	role
There are no results to be displayed.		

At the bottom of the interface, there's a footer with "DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom." and "Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018".

Total, count, and average per category — key analytical view for the dashboard.

5.5.2 Busiest transaction day

Purpose: Identify the date with the highest transaction count.

```
SELECT DATE(transaction_date) AS day, COUNT(*) AS txn_count
FROM transactions
GROUP BY DATE(transaction_date)
ORDER BY txn_count DESC
LIMIT 5;
```

The screenshot shows the DB Fiddle web application interface. At the top, there's a green banner with the text "Prep for your next tech interview with Exponent →". Below this is a blue header bar with navigation links: "Database: MySQL v8", "Run", "Save", "Load Example", "Collaborate", "Sign in", and "Have any feedback?".

The main interface is divided into three panels:

- Fiddle Title:** A text input field with "50 characters remaining." and a "Private Fiddle" toggle switch.
- Schema SQL:** Contains two SQL scripts:


```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60) NOT NULL UNIQUE,
13   description VARCHAR(255)
14 );
```
- Query SQL:** Contains a query:


```
1 -- 13: Attempt to DELETE Samuel Carter -- expect FK constraint error
2 (ERROR 1451)
3 -- fk_tp_user has no ON DELETE CASCADE, so MySQL blocks this delete
4 DELETE FROM users WHERE full_name = 'Samuel Carter';
```

Below the Schema SQL panel, there's a "Text to DDL" button. The Results panel shows a "Query Error: Cannot delete or update a parent row: a foreign key constraint fails ('test', 'transaction_participants', CONSTRAINT 'fk_tp_user' FOREIGN KEY (user_id) REFERENCES 'users' (user_id))".

At the bottom of the interface, there's a footer with "DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom." and "Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018".

Top 5 busiest days by transaction count.

5.5.3 Average fee per category

Purpose: Compute average transaction fee in each category — useful for fee analysis.

```
SELECT c.category_name, ROUND(AVG(t.fee), 2) AS avg_fee
FROM transactions t
JOIN transaction_categories c ON t.category_id = c.category_id
GROUP BY c.category_name
ORDER BY avg_fee DESC;
```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▶ Run Save Load Example Collaborate → Sign in Have any feedback? 🐦

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

design.dev
Design & Dev Tools
For Creators

Copy-paste ready CSS & JavaScript
patterns with accessibility baked in.

Schema SQL

```

1 CREATE TABLE users (
2   user_id      INT AUTO_INCREMENT PRIMARY KEY,
3   full_name    VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20)  UNIQUE,
5   user_type    ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id  INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60)  NOT NULL UNIQUE,
13   description  VARCHAR(255)
14 );

```

Text to DDL

Query SQL

```

1 -- 14: ANALYTICAL - Total amount transacted per category
2 SELECT
3   tc.category_name,
4   SUM(t.amount) AS total_amount_rwf,
5   COUNT(t.transaction_id) AS transaction_count
6 FROM transaction_categories AS tc
7 LEFT JOIN transactions AS t ON tc.category_id = t.category_id
8 GROUP BY tc.category_id, tc.category_name
9 ORDER BY total_amount_rwf DESC;

```

Results Copy as Markdown

Query #1 Execution time: 0.5ms

category_name	total_amount_rwf	transaction_count
Bank Deposit	50000.0	1
Incoming Money	25000.0	1
Payment to Code Holder	12000.0	2
Transfer to Mobile	10000.0	1

DB Fiddle - Crafted with 🍷 by Status200 in the United Kingdom. Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Average fee by category — Transfer to Mobile typically carries the highest average fee.

5.5.4 Top counterparties by amount received

Purpose: Rank users by total amount received across all incoming transactions.

```

SELECT u.full_name, SUM(t.amount) AS total_received
FROM transaction_participants tp
JOIN transactions t ON t.transaction_id = tp.transaction_id
JOIN users u      ON u.user_id        = tp.user_id
WHERE tp.role = 'receiver'
GROUP BY u.full_name
ORDER BY total_received DESC
LIMIT 5;

```

Prep for your next tech interview with Exponent →

Database: MySQL v8 ▶ ▶ Run Save Load Example Collaborate → Sign in Have any feedback? 

Fiddle Title

50 characters remaining.

Fiddle Description

300 characters remaining.

Private Fiddle **PRO**

This setting cannot be modified after saving the fiddle.

Upgrade to PRO

50% OFF for Early Adopters

Show Keyboard Shortcuts

 design.dev
Design & Dev Tools
For Creators

Copy-paste ready CSS & JavaScript patterns with accessibility baked in.

Schema SQL

```
1 CREATE TABLE users (  
2   user_id INT AUTO_INCREMENT PRIMARY KEY,  
3   full_name VARCHAR(100) NOT NULL,  
4   phone_number VARCHAR(20) UNIQUE,  
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT  
6   NULL DEFAULT 'CUSTOMER',  
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
8 );  
9  
10 CREATE TABLE transaction_categories (  
11   category_id INT AUTO_INCREMENT PRIMARY KEY,  
12   category_name VARCHAR(60) NOT NULL UNIQUE,  
13   description VARCHAR(255)  
14 );
```

Text to DDL

Query SQL

```
1 -- 15: ANALYTICAL - Busiest transaction day  
2 SELECT  
3   DATE(t.transaction_date) AS txn_day,  
4   COUNT(t.transaction_id) AS transactions_on_day  
5 FROM transactions AS t  
6 GROUP BY txn_day  
7 ORDER BY transactions_on_day DESC, txn_day DESC  
8 LIMIT 1;
```

Results

Query #1 Execution time: 0.32ms

Copy as Markdown

txn_day	transactions_on_day
2024-05-11	2

DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom.

Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018

Top 5 counterparties by total amount received — drives the leaderboard view in the dashboard.

6. Security and Integrity Rules

Beyond the foreign keys defined in `database_setup.sql`, the `security_integrity.sql` script applies additional rules that protect data accuracy and provide an audit trail. Each rule is shown below with the SQL that creates it and a screenshot demonstrating it in action.

6.1 CHECK constraints — reject impossible values

CHECK constraints prevent rows that would represent impossible business situations from ever entering the database.

```
ALTER TABLE transactions ADD CONSTRAINT chk_amount_positive
CHECK (amount >= 0);
```

```
ALTER TABLE transactions ADD CONSTRAINT chk_fee_non_negative
CHECK (fee >= 0);
```

```
ALTER TABLE transactions ADD CONSTRAINT chk_fee_le_amount
CHECK (fee <= amount);
```

The screenshot shows the DB Fiddle interface with the following content:

Schema SQL

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE transaction_categories (
11   category_id INT AUTO_INCREMENT PRIMARY KEY,
12   category_name VARCHAR(60) NOT NULL UNIQUE,
13   description VARCHAR(255)
14 );
```

Query SQL

```
1 -- 16: ANALYTICAL - Average fee per category, highest to lowest
2 SELECT
3   tc.category_name,
4   ROUND(AVG(t.fee), 2) AS avg_fee_rwf
5 FROM transaction_categories AS tc
6 JOIN transactions AS t ON tc.category_id = t.category_id
7 GROUP BY tc.category_id, tc.category_name
8 ORDER BY avg_fee_rwf DESC;
```

Results

category_name	avg_fee_rwf
Transfer to Mobile	100.0
Payment to Code Holder	62.5
Airtime Purchase	0.0
Bank Deposit	0.0

Attempting to INSERT a row with amount = -100 and the database rejecting it with a CHECK constraint violation.

These constraints catch programming errors and bad data early, at the storage layer rather than relying on application code.

6.2 Audit trigger — log every transaction insert

This AFTER INSERT trigger records every new transaction in `system_logs`, giving a tamper-evident audit trail without any application code involvement.

```
DELIMITER //
CREATE TRIGGER trg_transactions_audit
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
    INSERT INTO system_logs (transaction_id, log_level, log_message, processing_stage)
    VALUES (NEW.transaction_id, 'INFO',
            CONCAT('Transaction inserted: amount=', NEW.amount, ' RWF, ext_id=',
                IFNULL(NEW.external_txn_id, 'N/A')),
            'database');
END//
DELIMITER ;
```

The screenshot shows the DB Fiddle web application interface. The top bar includes a navigation link to Exponent, a database selector set to 'MySQL v8', and buttons for 'Run', 'Save', 'Load Example', and 'Collaborate'. On the right, there are links for 'Sign in' and 'Have any feedback?'. The main interface is divided into three panels:

- Fiddle Title:** A text input field with a character count (50 characters remaining).
- Fiddle Description:** A text input field with a character count (300 characters remaining).
- Private Fiddle:** A toggle switch currently set to 'PRO'.

The **Schema SQL** panel contains the following SQL:

```
1 CREATE TABLE users (
2   user_id INT AUTO_INCREMENT PRIMARY KEY,
3   full_name VARCHAR(100) NOT NULL,
4   phone_number VARCHAR(20) UNIQUE,
5   user_type ENUM('CUSTOMER', 'AGENT', 'MERCHANT', 'BANK', 'SYSTEM') NOT
6   NULL DEFAULT 'CUSTOMER',
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );
9 CREATE TABLE transaction_categories (
10  category_id INT AUTO_INCREMENT PRIMARY KEY,
11  category_name VARCHAR(60) NOT NULL UNIQUE,
12  description VARCHAR(255)
13 );
```

The **Query SQL** panel contains the following SQL:

```
1 -- 17: ANALYTICAL - Top 5 counterparties by total amount received
2 SELECT
3   u.full_name,
4   u.user_type,
5   SUM(t.amount) AS total_received_rwf,
6   COUNT(t.transaction_id) AS times_received
7 FROM users AS u
8 JOIN transaction_participants AS tp ON u.user_id = tp.user_id AND
9   tp.role = 'RECEIVER'
10 JOIN transactions AS t ON tp.transaction_id = t.transaction_id
11 GROUP BY u.user_id, u.full_name, u.user_type
12 ORDER BY total_received_rwf DESC
13 LIMIT 5;
```

The **Results** panel shows the output of the query, with an execution time of 0.45ms. The results are displayed in a table:

full_name	user_type	total_received_rwf	times_received
Samuel Carter	CUSTOMER	75000.0	2
Jane Smith	CUSTOMER	10000.0	1
Direct Payment Ltd	MERCHANT	4500.0	1
MTN Agent 54321	AGENT	1000.0	1

At the bottom of the interface, there is a footer with the text: "DB Fiddle - Crafted with ♥ by Status200 in the United Kingdom." and "Terms of Use • Privacy / Cookie Policy • Status200 Ltd © 2018".

After inserting a transaction, querying `system_logs` shows the trigger automatically wrote an audit row.

6.3 Duplicate-protection trigger

This BEFORE INSERT trigger blocks any attempt to insert a transaction with an `external_txn_id` that already exists, even before the UNIQUE constraint fires — useful for emitting a cleaner error message during ETL.

```
DELIMITER //
CREATE TRIGGER trg_block_duplicate_txn
BEFORE INSERT ON transactions
FOR EACH ROW
BEGIN
```



```

IF NEW.external_txn_id IS NOT NULL AND EXISTS (
  SELECT 1 FROM transactions WHERE external_txn_id = NEW.external_txn_id
) THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'Duplicate external_txn_id - transaction already imported.';
END IF;
END//
DELIMITER ;

```

6.4 v_transaction_summary view

A view that joins transactions, categories, participants, and users so downstream code can read complete transactions in one query without writing the join logic each time.

```

CREATE VIEW v_transaction_summary AS
SELECT
  t.transaction_id,
  t.external_txn_id,
  c.category_name,
  t.amount,
  t.fee,
  t.transaction_date,
  t.direction,
  MAX(CASE WHEN tp.role = 'sender' THEN u.full_name END) AS sender,
  MAX(CASE WHEN tp.role = 'receiver' THEN u.full_name END) AS receiver
FROM transactions t
JOIN transaction_categories c ON c.category_id = t.category_id
LEFT JOIN transaction_participants tp ON tp.transaction_id = t.transaction_id
LEFT JOIN users u ON u.user_id = tp.user_id
GROUP BY t.transaction_id, t.external_txn_id, c.category_name,
         t.amount, t.fee, t.transaction_date, t.direction;

```

6.5 v_daily_totals view

A view that pre-aggregates transactions by day. Used by the dashboard to render the time-series chart efficiently.

```

CREATE VIEW v_daily_totals AS
SELECT DATE(transaction_date) AS day,
       COUNT(*)              AS txn_count,
       SUM(amount)           AS total_amount,
       SUM(fee)              AS total_fees
FROM transactions
GROUP BY DATE(transaction_date);

```

8. Contributors

This document and the supporting code in the project repository are the product of the following team members' technical contributions:

Member	Role	Technical Contributions
Espoir	Database Design & Quality	ERD diagram and design rationale; CRUD and analytical test queries; query execution and result screenshots.
David	Database Engineer	database_setup.sql — schema DDL, primary and foreign keys, indexes, column comments, and seeded sample data; data dictionary content.
Gary	Security & Process	security_integrity.sql — CHECK constraints, triggers, and views; Scrum board and Agile workflow documentation.
Merci	Data Modeling	json_schemas.json — JSON entity schemas, complex nested transaction object, and SQL-to-JSON mapping documentation.

All individual technical contributions are also visible through commit history in the project's GitHub repository.